What Do AI Agents Actually Change?

An Empirical Taxonomy of Mutation Patterns in Performance-Improving Pull Requests

Illia Dovhoshliubnyi[1] Nima Soroush[1] Ashkan Sami[1] Alexander Brownlee[2]

SSBSE 2026 Challenge

[1]Edinburgh Napier University, Edinburgh, UK
illiadovho@gmail.com, {N.Soroush, A.Sami}@napier.ac.uk

[2]University of Stirling, Stirling, UK
alexander.brownlee@stir.ac.uk

AI Coding Agents: The Black Box

- **Autonomous Diffs**: Agents (Devin, Copilot, Cursor, Codex, Claude Code) submit PRs at scale, but their decision processes are proprietary and opaque.
- **The Diffs Speak**: We can't see *how* they think, but we can inspect *what* they change.
- **Our Goal**: Analyze successful performance-improving PRs from these agents to reverse-engineer their mutation behavior.

Why This Matters for SBSE & GI

- **Empirical Operators**: Genetic Improvement (GI) and Search-Based Software Engineering (SBSE) rely on mutation operators reflecting real-world changes.
- **The Gap**: Current taxonomies are derived from human-written patches.
- **Search Inefficiency**: Using human-derived operators on agent workflows wastes search budget on mutations agents never produce, while missing ones they rely on.

The Performance Scarcity Challenge

- **scarcity in numbers**: In the `AIDev-pop` dataset (33,596 total agent PRs):
- Only **324 PRs** carry a performance-related label ($< 1\%$).
- **Valuable Samples**: These rare successful cases provide a valuable window into how agents actually achieve performance improvements.

Research Questions

RQ1: Mutation Distribution
What syntactic mutation patterns characterise successful performance PRs from AI coding agents, and how do they differ from prior genetic improvement (GI) work?

RQ2: Contextual Variation
Do mutation patterns vary systematically across agent systems and across performance strategies?

Methodology & Classification Pipeline

- **Filtering Pipeline**: Start with 324 performance-labeled PRs $\rightarrow$ 280 retrievable diffs $\rightarrow$ 269 containing business-logic source hunks.
- **First-Pass Filter**: `claude-sonnet-4-6` retains hunks plausibly affecting runtime/memory/IO.
- **Dual-LLM Judge**: Independent classification by `claude-sonnet-4-6` and `gpt-5.4` (18 categories).
- **Intersection Agreement**: Full (48.4%) or partial (51.6%) agreement retained; full disagreements discarded. Yields final **1,254 hunks** across **216 PRs**.

RQ1: Overall Mutation Distribution

Three categories dominate the performance-improving changes:

1. **name_modification** (37.0%)
2. **object_creation** (26.4%)
3. **type_change** (22.7%)

Rare Mutations: Concurrency and error-handling modifications are very rare (e.g., *synchronization* at 0.8%, *inlining* at 0.2%), suggesting agents avoid high-risk runtime changes.

RQ1: Contrast with Traditional GI

How do AI agents differ from traditional/human GI mutation distributions?

Traditional GI Corpus

- Dominated by **no_change** (**84%**)
- Randomly sampled mutations

AI Agent Performance PRs

- **no_change** is completely absent (**0%**)
- Highly goal-directed, purposeful modifications

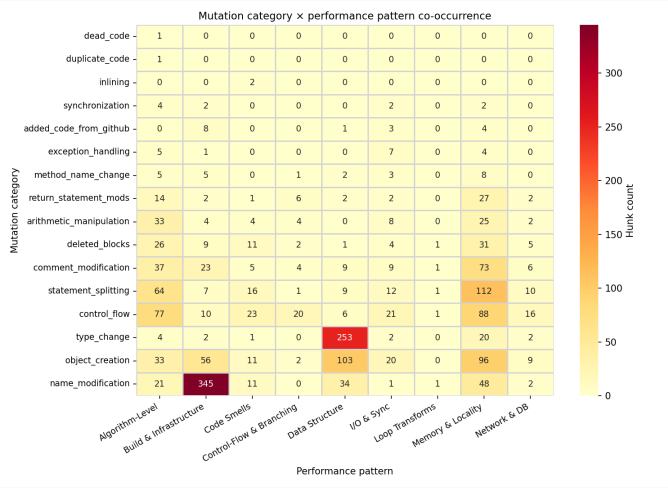
RQ1 Spotlight: The Devin "Carrier" Effect

- Why does **name_modification** (37.0%) lead so strongly?
- **Dependency Swapping**: In 78% of name modification cases, Devin swapped external libraries for faster alternatives (e.g., `chalk` → `picocolors`).
- **Carrier Category**: Name modification acts as a carrier category that spans imports, caching variable substitutions, and identifier renaming.

RQ2: Strategy-Driven Mutations

- Each performance strategy activates a largely disjoint subset of categories:
- **type_change** is 89% concentrated in *Data Structure* optimizations.
- **name_modification** is 75% concentrated in *Build & Infrastructure* (dependency swapping).
- **object_creation** is split between *Data Structure* (31%) and *Memory & Locality* (29%) (caching).

Visualizing Strategy Co-Occurrence



RQ2: Agent Mutation Vocabularies

Different agents have distinct mutation signatures:

- **Devin**: Dominated by *name_modification* (54%) (dependency swapping).
- **GitHub Copilot**: Led by *type_change* (38%) (data-structure upgrades).
- **OpenAI Codex**: Broadest structural profile; led by *control_flow* (24%) and *object_creation* (18%).
- **Cursor**: Elevated *comment_modification* (22%) (adds explanatory annotations alongside diffs).

Agent Mutation Profiles Table

Category	Devin	Copilot	Codex	Cursor
name_modification	**361**	30	61	11
object_creation	81	116	101	29
type_change	35	**224**	22	2
control_flow	51	48	**136**	24
statement_splitting	65	49	95	21
comment_modification	22	64	37	**41**
All categories (total)	*670*	*591*	*574*	*184*

Table 1: Top mutation categories by agent (counts from 1,254 hunks).

Implications for SBSE & GI

Priors for Reducing Search Space
By conditioning mutation operators on **agent identity** and **target optimization strategy**, we narrow the search space from 18 categories to a small handful.

- **Example 1**: Optimizing Data Structures? Limit operators to *type_change* and *object_creation*.
- **Example 2**: Optimizing Build/Infra? Focus on *name_modification* (dependency substitutions).
- Directly increases search efficiency in agent-assisted GI.

Threats to Validity & Limitations

- **Language Confound**: Agent and target language are correlated in the dataset (Devin $\rightarrow$ TypeScript, Copilot $\rightarrow$ Rust, Codex $\rightarrow$ Go); signatures reflect a mix of agent style and ecosystem.
- **Taxonomy Scope**: Extended *object_creation* to include module-level imports, explaining its high rate (26.4%) compared to traditional Java method-level scope.
- **Merged vs. Unmerged**: Includes both merged (324 hunks) and unmerged (927 hunks). Top categories overlap strongly ($\rho = 0.90$), but *control_flow* and *statement_splitting* are $\sim 3\times$ more prevalent in merged hunks.
- **Judge Limitations**: LLM accuracy is bounded ($\sim$67.5%); using the intersection strategy ensures conservative lower bounds.

Conclusion

- **Legibility**: Opaque AI agent contributions can be reverse-engineered using SBSE mutation theory.
- **Structural Specificity**: Agent performance PRs focus on name modifications, object creations, and type changes—unlike traditional GI's flat distributions.
- **Empirical Priors**: Context-aware operator selection (knowing the agent and optimization target) drastically reduces the SBSE mutation space.